EXPLORANDO A LÓGICA DE PROGRAMAÇÃO ATRAVÉS DE UM JOGO COM AUXÍLIO DA REALIDADE AUMENTADA

Bruno Ricardo Junkes, Dalton Solano dos Reis – Orientador

Curso de Bacharel em Ciência da Computação
Departamento de Sistemas e Computação
Universidade Regional de Blumenau (FURB) – Blumenau, SC – Brasil

brjunkes@furb.br, dalton@furb.br

Resumo: *Este artigo apresenta um jogo para sistemas Android e iOS, com o intuito de auxiliar no aprendizado da lógica de programação, tendo maior foco nas fases iniciais de ensino. O jogo foi desenvolvido utilizando o motor de jogos Unity juntamente ao framework AR Foundation, que permite trabalhar com o conceito de Realidade Aumentada (RA). Conceito esse que foi utilizado com o objetivo de criar uma maior imersão dos cenários propostos. Através de blocos com comandos, o jogador deve montar um algoritmo, conjunto de instruções que devem ser executadas, indicando qual caminho os personagens devem seguir para chegar ao seu destino, desviando de obstáculos ou interagindo com objetos espalhados pelo caminho. Foram realizados testes com um público que possuía algum conhecimento em lógica de programação, com o intuito de avaliar sua receptividade e eficácia. Os objetivos foram atingidos, visto que os jogadores conseguiram interagir com a RA e compreender o funcionamento de cada bloco, concluindo assim boa parte dos níveis propostos.*

Palavras-chave*: Lógica de Programação. AR Foundation. Realidade Aumentada. Algoritmo.*

1 INTRODUÇÃO

É notável que nos últimos anos, uma das áreas que mais tem crescido é a da computação e com isso, exigindo cada vez mais profissionais qualificados para atuar em suas diversas subáreas. Contudo, sabe-se que esta não é uma área fácil, trazendo uma alta complexidade consigo e fazendo com que muitos tenham dificuldades em absorver conteúdos relacionados.

Devido a estas dificuldades, acaba-se culminando na desistência por grande parte dos estudantes de computação, muitas vezes resultante de metodologias defasadas de ensino utilizados por seus professores, complexidade da lógica de programação e falta de conhecimento de conceitos básicos (Souza; Batista; Barbosa, 2016). Morais, Mendes Neto e Osório (2020, p. 3) também apontam que "cada aluno possui suas dificuldades individuais, seu ritmo de aprendizagem, seus interesses e motivações, de forma que os professores precisam identificar as características de seus alunos e as dificuldades por eles enfrentadas para que consigam dar o suporte devido.".

Tendo estas dificuldades em mente, faz-se necessária a criação ou adaptação de métodos utilizados para lecionar as matérias que envolvam a área da computação, a fim de "proporcionar um ensino e aprendizagem de programação mais consistente e atraente para os alunos, inclusive com a inserção da programação na educação básica, na intenção de que o aluno tenha acesso a conteúdos da computação." (Souza; Falcão; Mello, 2021, p. 1265). Um conceito importante não só para a área, mas também para tarefas do cotidiano, que deve ser abordado é o da lógica de programação.

A lógica de programação pode ser entendida como um conjunto de regras e técnicas utilizadas para criar algoritmos, ou seja, sequências lógicas de passos que visam resolver determinados problemas ou realizar tarefas específicas. Segundo Batista, Fonseca e Horta (2020, p. 99) o ensino da lógica "beneficia o desenvolvimento do processo de aprendizagem, como o raciocínio lógico e a tomada de decisão, auxiliando na formação de ideias e resolução de problemas.".

Uma área que vem crescendo na computação, trazendo consigo um novo horizonte para o desenvolvimento de novas metodologias que visam auxiliar no ensino da lógica de programação, é a da Realidade Aumentada (RA). A RA nada mais é, do que a combinação de elementos do mundo real e do virtual em tempo real, possibilitando uma melhora nos sentidos, principalmente da visão (Elvas, 2018). Usar a tecnologia da RA neste contexto é favorável pois cria um ambiente descontraído, usando uma tecnologia relativamente nova, que chama a atenção e desperta o interesse das pessoas que interagem com ela. Não só isso, mas também sua facilidade de uso, pois, para executar uma simples aplicação é necessário apenas um dispositivo que possua uma câmera, que nos dias de hoje se tornou algo comum com o uso de dispositivos como o celular.

Com base no exposto, este trabalho apresenta um jogo que poderá ser usado como ponto de partida para o estudo da lógica nas fases iniciais de ensino e servir como uma ferramenta para estudantes da área da computação. O objetivo principal deste trabalho é disponibilizar um jogo para as plataformas Android e iOS que seja capaz de introduzir e/ou melhorar o desenvolvimento da lógica de programação, tanto para crianças quanto para pessoas que já possuem algum conhecimento. Os objetivos específicos são: disponibilizar um ambiente em RA sem a necessidade de marcadores para

fixação dos cenários, os quais devem permitir a interação com objetos 3D e coletar a opinião dos usuários que interagirem com o jogo na intenção de avaliar seu funcionamento e proposta.

2 FUNDAMENTAÇÃO TEÓRICA

Nesta segunda seção será apresentado os conceitos para o entendimento do projeto realizado. Na subseção 2.1, tem-se uma introdução aos conceitos de lógica de programação e algoritmos. Com a subseção 2.2, apresenta-se o conceito de Realidade Aumentada (RA). A subseção 2.3 descreve as principais características do *framework* AR Foundation. Por fim, na subseção 2.4 são listados os trabalhos correlatos.

2.1 LÓGICA DE PROGRAMAÇÃO E ALGORITMOS

Segundo Santos (2015) a lógica é um campo amplo de estudo que abrange o raciocínio e as leis do pensamento válido. Ela pode ser aplicada em atividades diversas e nas áreas das Ciências Exatas e Matemática, sendo essencial para profissionais que buscam solucionar problemas e atingir metas eficientemente, utilizando recursos computacionais. Santos (2015) também aponta que o conhecimento em lógica é crucial para lidar com problemas administrativos, de controle, planejamento e estratégia, exigindo habilidades de raciocínio sólido. Já a lógica de programação conforme Xavier (2014, p. 21) "significa apenas contextualizar a lógica na programação de computadores, buscando a melhor sequência de ações para solucionar determinado problema.".

"Muitos recursos são usados pelos profissionais para representar o seu raciocínio lógico como os fluxogramas, os diagramas de bloco e até mesmo a linguagem natural e variações da língua nativa do profissional" (Santos, 2015, p. 26). Ainda segundo Santos (2015), a lógica de programação pode ser codificada em linguagens de programação seguindo padrões. Sendo a mais conhecida e utilizada, a programação estruturada é uma técnica importante que visa agilizar o desenvolvimento, facilitar a depuração, verificar erros e facilitar a manutenção dos programas. Ela envolve criar sequências de comandos, combinar blocos de instruções, compartilhar módulos e realizar revisões em equipe.

A lógica de programação está fortemente relacionada ao conceito de algoritmos, uma vez que ambos tem como premissa gerar sequências de passos lógicos e instruções que representam a solução para um problema específico. Segundo Santos (2015) um algoritmo pode ser representado de várias formas, mas o importante é que seja compreensível e de fácil interpretação, na qual a clareza na descrição dele facilite sua depuração e correção de erros.

2.2 REALIDADE AUMENTADA

Segundo Tori, Hounsell e Kirner (2020) a ideia de aumentar a realidade utilizando recursos como espelhos, lentes e iluminações devidamente posicionados tem sido trabalhado desde o século XVII, porém, foi a partir de 1968 que o primeiro protótipo utilizando o conceito de RA foi criado por Ivan Sutherland, na qual permitia juntar imagens 3D geradas em computador sobre imagens reais. Embora diversas pesquisas abordando a RA já estavam sendo feitas, o termo somente foi desvinculado do conceito de Realidade Virtual (RV), que trata da completa imersão do usuário em um mundo virtual, em 1992 com o artigo de Tom Caudell e Tom Mizell, sendo em 2017 atribuído a Tom Caudell a criação do termo RA.

A RA é uma das subáreas da computação que se beneficiou da evolução da Tecnologia da Informação (TI), se tornando cada vez mais popular e acessível. Ela busca renderizar objetos virtuais 2D e/ou 3D no mundo real, permitindo com que as pessoas interajam com estes ao mesmo tempo que interagem com objetos do mundo real. Segundo Kirner e Siscoutto (2007), a RA pode ser definida como um enriquecimento do mundo real, utilizando-se de elementos virtuais como textos, imagens dinâmicas e modelos 3D, sendo gerados por dispositivos tecnológicos em tempo real, com isso, permitindo que o usuário se sinta no mundo real, podendo interagir com elementos do mundo virtual.

Dependendo do contexto que se pretende utilizar a RA, pode-se ter dois tipos de entradas: baseada em visão e baseada em sensores. A RA baseada em visão é mais robusta, flexível e fácil de utilizar, porém, possui problemas com iluminação e oclusão de informações. É nela que se encontra o recurso mais conhecido dentro da área, os marcadores, elementos utilizados para fornecer pontos de referência para a sobreposição dos objetos virtuais no mundo real. Já a RA baseada em sensores é mais precisa, utilizando sensores presentes nos dispositivos para fornecer informações e interações virtuais no mundo real, tendo incluso recursos como acelerômetros, giroscópios, Sistemas de Posicionamento Global (Global Positioning System – GPS), entre outros (Tori; Hounsell; Kirner, 2020).

2.3 AR FOUNDATION

É possível integrar a RA em dispositivos móveis usando um *framework* desenvolvido especificamente para esta tecnologia e seu Kit de Desenvolvimento de Software (Source Development Kit - SDK), um conjunto de ferramentas, bibliotecas e documentações que possuem suas próprias especificações em que sua escolha é feita de acordo com critérios específicos. O AR Foundation é um exemplo de framework desenvolvido com este propósito.

Através do AR Foudation é possível criar experiências de RA para as plataformas Android e iOS a partir do motor de jogos Unity. Com esta tecnologia é fornecida uma Interface de Programação de Aplicativos (Application

Programming Interface - API) que une os principais recursos de RA como ARKit, ARCore, Magic Leap e HoloLens, possibilitando o rastreamento de imagens e objetos 3D, a detecção de planos e o reconhecimento facial (Unity, 2023).

2.4 TRABALHOS CORRELATOS

A seguir são apresentados três trabalhos correlatos. No Quadro 1 é apresentado o projeto de Saraiva (2022), que busca ajudar no treinamento dos pilares do Pensamento Computacional (PC). No Quadro 2 é apresentado o trabalho de Mello e Antoniazzi (2020), abordando um jogo em RA, que tem como objetivo trazer de forma divertida os conceitos de lógica e programação. Por fim, no Quadro 3 é descrito o trabalho de Kuntz (2020), que traz a ideia de uma Interface de Usuário Tangível (IUT) com o intuito de trabalhar o conceito de PC utilizando como base o Furbot.

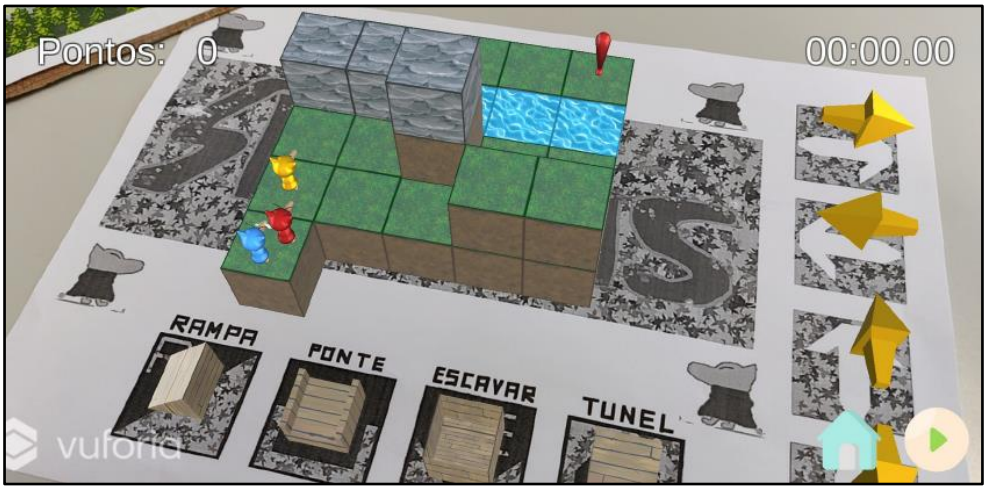
Quadro 1 - Building a game with Augmented Reality: for training computational thinking

Referência	Saraiva (2022)
Objetivos	O jogo tem como objetivo ajudar a desenvolver os quatro pilares do PC: reconhecimento de padrões, decomposição, algoritmos e abstração.
Principais funcionalidades	Utilização de cartões físicos que representam ações que um determinado personagem pode executar, para construir um caminho lógico na tentativa de guiar todos os personagens ao seu destino.
Ferramentas de desenvolvimento	Para o desenvolvimento do jogo foi utilizada a Unity, configurada para sistemas Android. Já para criar a visualização em RA utilizou-se a plataforma Vuforia, que realiza o processamento de imagens gerando marcadores virtuais. Outra ferramenta utilizada foi o Blender, que permite criar animações, efeitos e modelos 3D.
Resultados e conclusões	O objetivo de desenvolver um jogo em RA que colaborasse com o estudo do PC foi alcançado e que a partir dos estudos, pesquisas, apresentação e *feedbacks* coletados é realmente possível treinar habilidades de PC através de jogos.

Fonte: elaborado pelo autor (2023).

A Figura 1 traz uma visão detalhada do jogo proposto, destacando um dos cenários disponíveis. Nessa representação visual são apresentados os personagens do jogo, juntamente com uma série de cartões que desempenham um papel fundamental na criação do percurso. Esses cartões incluem construções, como rampas, pontes e túneis, bem como setas direcionais que tem como objetivo guiar os movimentos dos personagens, indicando claramente a direção que devem seguir ao longo do jogo.

Figura 1 - Exemplo de cenário e cartões das ações



Fonte: Saraiva (2022).

Quadro 2 - Jogo com utilização de realidade aumentada voltado para o desenvolvimento lógico aplicado ao ensino fundamental e médio

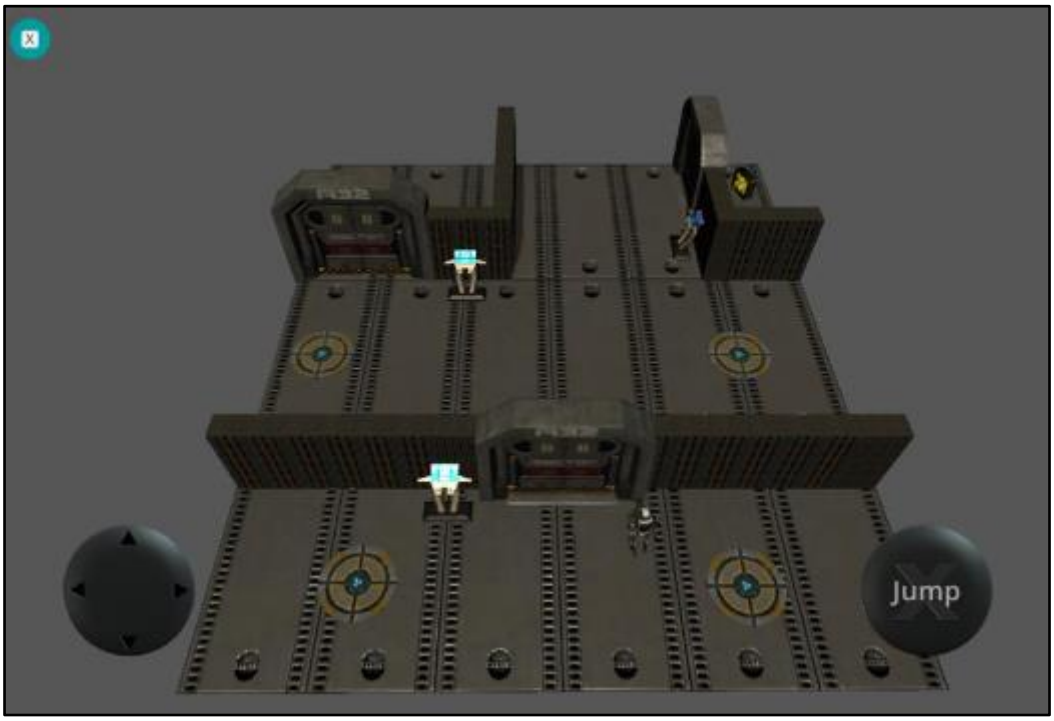
Referência	Mello e Antoniazzi (2020)
Objetivos	Criar uma metodologia de ensino que proporcione um meio mais divertido e desafiador para estudantes sobre lógica e programação e aumentar o interesse nas áreas da informática, fornecendo um acesso facilitado ao estudo desses assuntos.
Principais funcionalidades	Apresentação de conceitos relacionados a lógica e algoritmos no início de cada fase e *joysticks* para movimentação livre do personagem pelos cenários afim de resolvê-los.
Ferramentas de desenvolvimento	Para o desenvolvimento do jogo foi utilizada a Unity, configurada para sistemas Android. Utilizou-se a plataforma Asset Store para a coleta de modelos 3D para os cenários e personagem. Por fim, foi utilizado também a plataforma Vuforia para a criação dos marcadores virtuais.

Resultados e conclusões	Atualmente o jogo se encontra em estado parcial de desenvolvimento, na qual serão criadas mais fases, um aprimoramento das texturas e iluminação e novos testes a fim de detectar possíveis erros. Com sua conclusão, planeja-se apresentar o jogo em escolas com o intuito de aplicar questionários, no qual poderá ser avaliada a eficácia dele na apresentação dos conceitos envolvidos na programação.

Fonte: elaborado pelo autor (2023).

A partir da Figura 2 é possível visualizar um dos cenários disponíveis no jogo Robo Kyle, na qual pode-se observar os *joysticks* utilizados para a movimentação do personagem pelo mapa, juntamente com os painéis que devem ser ativados de forma lógica para abrir as portas por onde Kyle deve passar a fim de finalizar o nível.

Figura 2 - Cenário do jogo Robo Kyle



Fonte: Mello e Antoniazzi (2020).

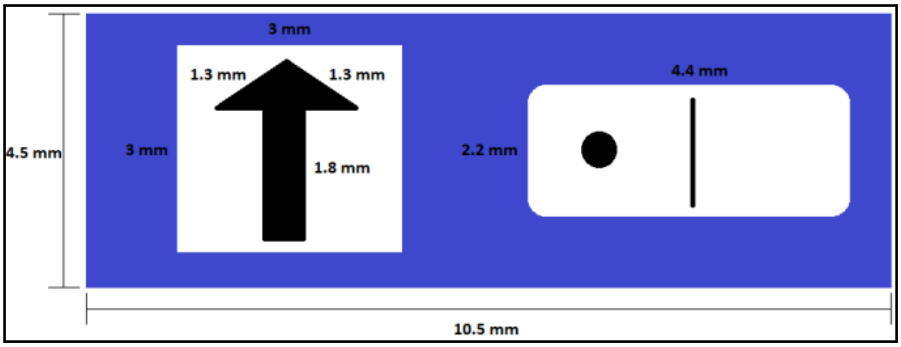
Quadro 3 - Interface de usuário tangível para trabalhar com o pensamento computacional no FURBOT

Referência	Kuntz (2020)
Objetivos	Criar um módulo de IUT para o jogo Furbot com o intuito de treinar o PC.
Principais funcionalidades	Detecção do contorno de imagens para “traduzi-las” em comandos para o personagem principal.
Ferramentas de desenvolvimento	Para o desenvolvimento do jogo foi utilizada a Unity juntamente ao pacote OpenCV For Unity que oferece métodos para detecção de imagem. Para criação das peças que serão interpretadas pelo jogo, foi utilizado o software de edição de imagens Photoshop.
Resultados e conclusões	Embora ainda existam limitações, como influência da luz ambiente, sombras, posição e ângulo das peças, o trabalho atingiu seus objetivos, apresentando uma aplicação Android que reconhece e executa comandos com base nas peças criadas pelos usuários.

Fonte: elaborado pelo autor (2023).

Furbot é um projeto desenvolvido pelos alunos da Universidade Regional de Blumenau (FURB), que tem como objetivo desenvolver o PC através de jogos digitais, estimulando o desenvolvimento de crianças em níveis iniciais de ensino (FURB, 2022). Quanto ao trabalho de Kuntz (2020), tem-se na Figura 3 uma demonstração do modelo de peça criada, com as dimensões pré-requisitadas para que o jogo consiga interpretá-las corretamente. Também é possível notar uma grande semelhança com as peças do jogo conhecido como dominó, que segundo Kuntz (2020), foi criado intencionalmente desta forma para que se possa utilizar peças reais como instruções para o jogo. Já na Figura 4 tem-se um dos cenários presentes no jogo, juntamente com as instruções interpretadas com o uso da IUT desenvolvida logo a direita.

Figura 3 - Modelo de peça utilizado para a IUT



Fonte: Kuntz (2020).

Figura 4 - Cenário de exemplo com comandos interpretados pela IUT

Fonte: Kuntz (2020).

3 DESCRIÇÃO DO JOGO

Na presente seção serão descritos os detalhes de especificação e implementação do jogo desenvolvido nomeado como “ProgramAR”, a qual está dividida em três subseções. Na subseção 3.1 é apresentada uma visão geral dos elementos presentes no jogo, com a subseção 3.2 tem-se a especificação, trazendo os quadros de requisitos funcionais e não funcionais e o fluxograma dos principais processos. Já na subseção 3.3 são abordados detalhes das implementações das funcionalidades presentes no jogo.

3.1 VISÃO GERAL

O jogo ProgramAR traz ao jogador desafios que envolvem criar uma sequência lógica de comandos para construir um caminho, essa sequência é conhecida como algoritmo que o jogo introduz como um mapa. Esse mapa tem como objetivo guiar os personagens, dois astronautas perdidos em outro planeta, de volta para o seu foguete.

Para a construção dos mapas cada astronauta tem acesso à um painel próprio, em que o jogador pode adicionar diversos blocos com comandos, cada um representando uma ação que pode ser executada pelos astronautas, são elas: andar um bloco a frente, rotacionar para a direita ou esquerda, interagir com os objetos que estiverem a frente e esperar, que mantém um astronauta parado até que o outro finalize sua próxima ação. Além disso, o jogo trabalha com a introdução ao conceito de funções de programação, trazendo níveis que exigem que o jogador utilize sequências de comandos em um painel específico para funções, que possam ser repetidas várias vezes em pontos distintos de seu algoritmo.

Durante o desenvolvimento do mapa, o jogador também deve se ater aos desafios propostos pelos cenários. Desafios esses que podem envolver apenas um astronauta específico, como coletar uma chave para abrir uma porta que está bloqueando seu caminho, ou envolver a colaboração dos dois astronautas, como um deles puxar uma alavanca para abrir a porta que está bloqueando o outro ou até mesmo manter um botão pressionado para desativar espinhos que matariam o astronauta que passar por cima.

Como um meio de ajudar o jogador quando não estiver conseguindo construir uma lógica que consiga levar os astronautas até o foguete, o jogo também conta com uma tela de *logs* de erro. Essa tela informa ao jogador as ações que os astronautas tentaram executar e não conseguiram juntamente a descrição dos possíveis motivos, assim dando uma ideia de quais comandos o jogador deve alterar para consertar sua lógica.

O jogo divide todas estas funcionalidades em duas páginas de níveis, trazendo na primeira uma introdução as ações básicas já citadas e a segunda o conceito de funções, juntamente a uma breve explicação sobre *loops* infinitos, estruturas de repetição que não possuem um fim definido. Cada conceito é explicado detalhadamente a partir de tutoriais, onde um personagem não jogável chamado Iara conversa com o jogador explicando o objetivo do jogo, como utilizar os blocos com comandos e como cada ação é executada pelos astronautas. Com o APÊNDICE A é possível ter uma visão destas e de todas as outras telas do jogo, juntamente ao uso da visualização em RA com os cenários, personagens e desafios citados.

3.2 ESPECIFICAÇÃO

Esta seção apresenta a arquitetura do aplicativo. O Quadro 4 apresenta os Requisitos Funcionais (RF), já o Quadro 5 apresenta os Requisitos Não Funcionais (RNF).

Quadro 4 - Requisitos funcionais

RF01: permitir que o jogador navegue entre os menus do jogo
RF02: permitir a locomoção em volta do cenário que compõe os níveis com o intuito de aumentar a interatividade com o jogo
RF03: permitir a interação com as entidades dos níveis a partir do dispositivo móvel
RF04: permitir que seja possível montar algoritmos utilizando os blocos com comandos com o intuito de desenvolver a lógica de programação
RF05: permitir que seja possível visualizar os blocos que compõem o código de cada entidade
RF06: permitir que seja possível visualizar os *logs* de erro que impediram o sucesso do nível
RF07: permitir que os blocos do código das entidades sejam reprogramados
RF08: permitir executar os códigos programados com os blocos, para visualizar em tempo real a lógica desenvolvida

Fonte: elaborado pelo autor (2023).

Quadro 5 - Requisitos não funcionais

RNF01: ser desenvolvido no motor de jogos Unity com uso do *framework* AR Foudation
RNF02: ser desenvolvido para as plataformas móveis Android e iOS
RNF03: renderizar os níveis em qualquer superfície plana controlada
RNF04: salvar o progresso do jogador através da classe `PlayerPrefs`
RNF05: utilizar modelos para construção do jogo retirados do site Kenney

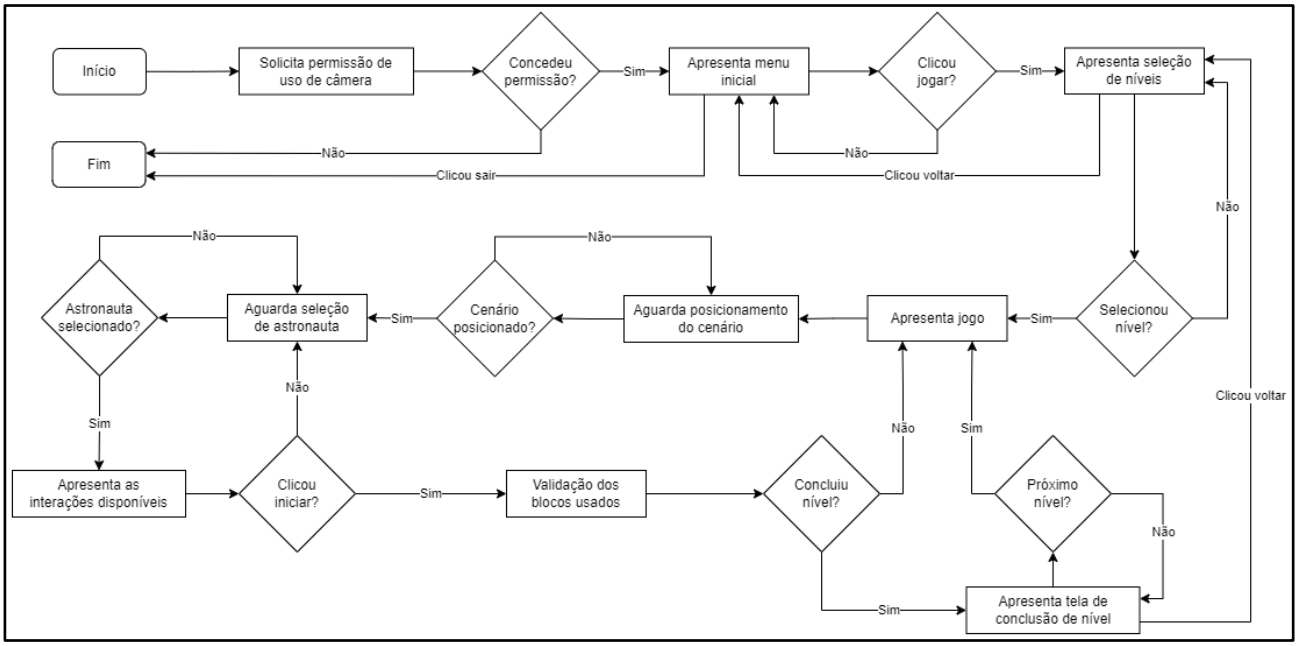
Fonte: elaborado pelo autor (2023).

O fluxograma dos processos do jogo apresentado na Figura 5, foi construído utilizando a ferramenta online Draw.io, disponível gratuitamente. Com o início do fluxograma, é solicitado ao usuário a permissão de uso da câmera do aparelho, necessária para que a realidade aumentada possa ser utilizada, caso contrário, o jogo não poderá montar os cenários e assim, nenhuma das interações ficarão disponíveis para o jogador. Com a permissão concedida, o jogador será redirecionado para o menu inicial, que lhe permitirá sair do jogo ou iniciá-lo, sendo redirecionado para a tela de seleção de níveis, que por sua vez permitirá voltar para o menu inicial ou selecionar um nível. Quanto a seleção de níveis, existem atualmente duas páginas, a primeira contendo níveis que apresentaram os comandos básicos do jogo (andar, girar, interagir e esperar) e a segunda trazendo o conceito de função.

Ao selecionar um nível, a tela principal do jogo é aberta e apresentará o cenário proposto ao jogador, que deverá mover o aparelho e clicar na tela para selecionar o local que deseja fixar este cenário. Tendo o posicionado, será possível selecionar um dos astronautas apontando a câmera para eles e clicando na tela, o que abrirá a tela de programação e apresentará os blocos com comandos disponíveis para o nível. Tendo montando sua lógica, o jogador poderá executar os comandos a partir do botão iniciar, que então irá traduzir os blocos com comandos em funções internas e validará se ambos os astronautas chegaram no destino. Em caso positivo, será apresentada a tela de conclusão de níveis, que permitirá voltar a tela de seleção de níveis ou avançar para o próximo caso exista. Durante a execução dos comandos, o jogador poderá visualizar os astronautas andando pelo cenário, para que em casos de falha, ele possa entender melhor onde errou.

Outra interação que ficará disponível ao clicar em um astronauta, caso o jogador já tenha executado os comandos pelo menos uma vez e não tenha concluído o nível, é o botão de *logs*. Assim que ele é pressionado será aberta uma tela de *log* de erros com informações dos comandos que falharam ao serem executados pelos astronautas, visando auxiliar o jogador a descobrir por que sua lógica falhou e o nível não ter sido concluído.

Figura 5 - Fluxograma dos processos do jogo



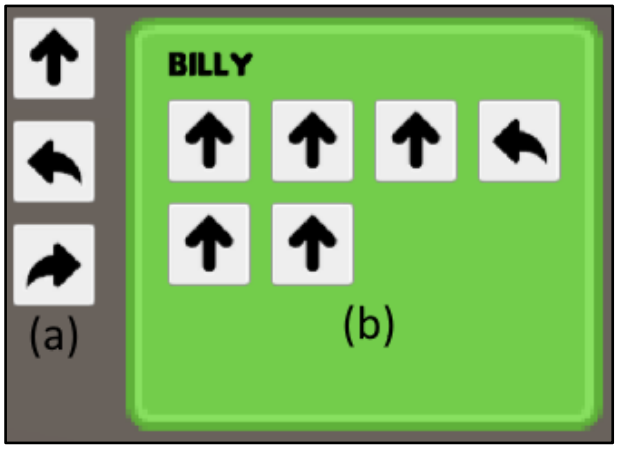
Fonte: elaborado pelo autor (2023).

3.3 IMPLEMENTAÇÃO

Para a implementação do jogo foi utilizado o motor de jogos Unity na versão 2021.3.11f1 com sua linguagem de programação padrão C# e quanto ao *framework* AR Foundation, foi utilizada a versão 5.1. O desenvolvimento foi dividido em três partes, sendo elas a implementação dos blocos com comandos e *logs* de erro, validação da lógica desenvolvida e a visualização dos cenários com a Realidade Aumentada (RA).

Para controlar os blocos selecionados pelo jogador e da lista de *logs* de erro para cada astronauta, foi criada uma classe chamada `Algorithm`. Com relação aos blocos, caso o jogador selecione uma das opções de blocos disponíveis (Figura 6 (a)) será adicionado um novo identificador dentro de uma lista de controle nesta classe, ao mesmo tempo que se pressionado um bloco da lista (Figura 6 (b)), o identificador correspondente será removido da lista de controle. Para a lista de *logs* de erro, os itens são adicionados conforme os comandos dos blocos são executados e não conseguem concluir a ação correspondente, como no exemplo da Figura 7 retirado do código fonte do jogo, em que o astronauta tentou interagir com uma porta sem possuir uma chave. Para cada caso, é adicionada uma mensagem específica com o correspondente passo que falhou.

Figura 6 - Tela de programação de um astronauta



Fonte: elaborado pelo autor (2023).

Figura 7 – Exemplo de log de erro

```
194        if (hit.transform.tag == "Door")
195        {
196            if (!haveKey)
197            {
198                log.Add($"Passo {idCurrentCommand} - Não peguei a chave");
199                return;
200            }
201
202            hit.transform.parent.GetComponentInChildren<Animator>().enabled = true;
203        }
```

Fonte: elaborado pelo autor (2023).

A classe `Algorithm` também é responsável por interpretar os comandos presentes na lista de controle, a qual será iterada e para cada identificador, será feita uma "tradução" para chamar uma função que tentará executar a ação correspondente. Cada uma dessas funções trabalha com atributos dos astronautas que são contidos pelo tipo `GameObject`, componente da Unity que representa qualquer elemento presente na cena de um jogo. Exemplo disto são os métodos `RotateRight` e `RotateLeft` vistos na Figura 8, que modificam o atributo `Rotation` do objeto afim de rotacionar o eixo y dos astronautas na direção selecionada.

Figura 8 - Métodos de rotação dos astronautas

```
163    private void RotateRight()
164    {
165        float playerYRot = this.gameObject.transform.rotation.eulerAngles.y;
166        this.gameObject.transform.rotation = Quaternion.Euler(0, playerYRot + 90, 0);
167    }
168
169    private void RotateLeft()
170    {
171        float playerYRot = this.gameObject.transform.rotation.eulerAngles.y;
172        this.gameObject.transform.rotation = Quaternion.Euler(0, playerYRot - 90, 0);
173    }
```

Fonte: elaborado pelo autor (2023).

Para validar o algoritmo criado pelo jogador, a classe `GameControll` foi implementada. Nela, é feita uma constante validação através do método `Update` da própria Unity que é chamado a cada quadro executado pelo jogo, com o objetivo de verificar se ambos os astronautas concluíram todos os comandos presentes em sua lista de controle. Em caso positivo, também será validado se eles chegaram no objetivo, no qual é lançado um `Raycast`, raio traçado de um ponto a outro invisível para o jogador, da posição do personagem até o bloco de cenário a sua frente com o intuito de verificar se há um `GameObject` chamado "Rocket". Se esta validação retornar verdadeiro, é apresentado ao jogador a tela de conclusão de nível e o progresso é salvo através da classe `PlayerPrefs`, que é uma das formas que a Unity utiliza para salvar dados dos jogos, caso contrário será necessário reiniciar o nível e modificar o algoritmo.

Com a classe `ScenePlacement` implementada foi possível trabalhar com as configurações do AR Foundation e criar a visualização em RA, utilizando as informações de `ARRaycastManager` e `Pose`. `ARRaycastManager` é o responsável pela interação com os subsistemas de RA do Unity (ARKit e ARCore para desenvolvimento iOS e Android respectivamente) para realizar as detecções das superfícies no mundo real, lançando `Raycast's` em RA. Já o `Pose` é responsável por armazenar a posição e a orientação de objetos virtuais no mundo real. Conforme Figura 9, é possível visualizar o uso do `ARRaycastManager` para receber todas as colisões detectadas filtradas como plano, ou seja, apenas colisões com objetos reais como pisos, mesas ou paredes. É possível visualizar também o uso de `placementPose`, variável do tipo `Pose`, recebendo as informações da primeira colisão válida na lista, para posteriormente utilizar na ancoragem do cenário selecionado pelo jogador a partir de um clique na tela do aparelho.

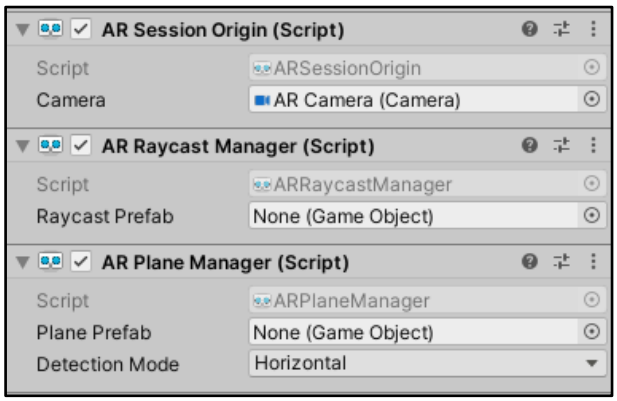
Figura 9 - Uso do ARRaycastManager e Pose

```
46    private void UpdatePlacementPose()
47    {
48        Vector3 screenCenter = Camera.current.ViewportToScreenPoint(new Vector3(0.5f, 0.5f));
49        List<ARRaycastHit> hits = new List<ARRaycastHit>();
50        arRaycast.Raycast(screenCenter, hits, TrackableType.Planes);
51
52        placementPoseValid = hits.Count > 0;
53        if (placementPoseValid)
54            placementPose = hits[0].pose;
55    }
```

Fonte: elaborado pelo autor (2023).

Além desta classe, também é necessário realizar a configuração de alguns componentes na própria cena da Unity, como apresentado na Figura 10. Para gerenciar a relação entre o espaço do mundo real e virtual é necessário um `ARSessionOrigin`, juntamente a uma `ARCamera` vinculada para realizar o acesso a câmera do aparelho em uso. Também são necessários um `ARRaycastManager` e um `ARPlaneManager` responsável por detectar os planos no ambiente do mundo real e que neste caso está configurado para somente planos horizontais, ignorando assim planos como paredes por exemplo.

Figura 10 - Componentes necessários para usar a RA



Fonte: elaborado pelo autor (2023).

Ainda para a parte de RA, na classe `GameControll` foi implementado o método `SelectAstro`, que valida se o jogador pressionou a tela do aparelho em uso e a partir da posição do toque, lança um `Raycast` até detectar a primeira colisão. Caso esta colisão seja com um objeto que possua a *tag* `Player`, que representa os astronautas, uma borda vermelha será criada a sua volta e também será armazenada uma referência para ele (Figura 11), liberando assim as interações de programação e *log* de erro.

Figura 11 - Seleção de um astronauta

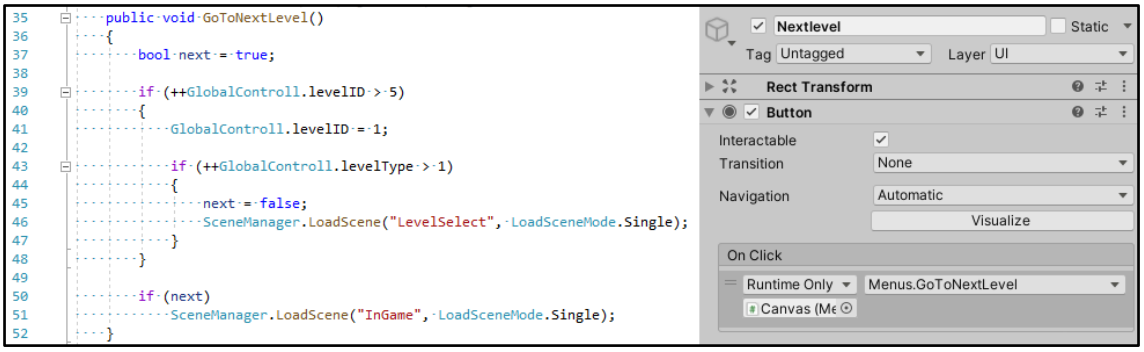
```
127        if (Input.touchCount > 0)
128        {
129            Ray ray = Camera.main.ScreenPointToRay(Input.GetTouch(0).position);
130            if (Physics.Raycast(ray, out raycastHit))
131            {
132                highlight = raycastHit.transform;
133                if (highlight.CompareTag("Player") && highlight != selection)

144        highlight.gameObject.GetComponent<Outline>().OutlineColor = Color.red;
145        highlight.gameObject.GetComponent<Outline>().OutlineWidth = 7.0f;
```

Fonte: elaborado pelo autor (2023).

Para as trocas entre as telas disponíveis no jogo, foi criada uma classe chamada `Menus`. Nela estão presentes os métodos que carregam as cenas utilizando a classe `SceneManager` da Unity, que permite realizar diversas interações com as cenas previamente configuradas. Um exemplo é o exposto na Figura 12 (a), na qual caso seja pressionado o botão vinculado ao método `GoToNextLevel`, como pode ser visto na propriedade `OnClick` da Figura 12 (b), será feita uma verificação se há um próximo nível, seja na página atual ou em uma próxima, para carregar o novo nível ou em caso de inexistência, carregar o menu de seleção de níveis. Esta classe também é responsável por dar início a execução da lógica desenvolvida e reiniciar o nível, caso o jogador pressione o botão `Play/Stop` presente na interface principal do jogo.

Figura 12 - Exemplo de uso de botões na Unity



Fonte: elaborado pelo autor (2023).

4 RESULTADOS

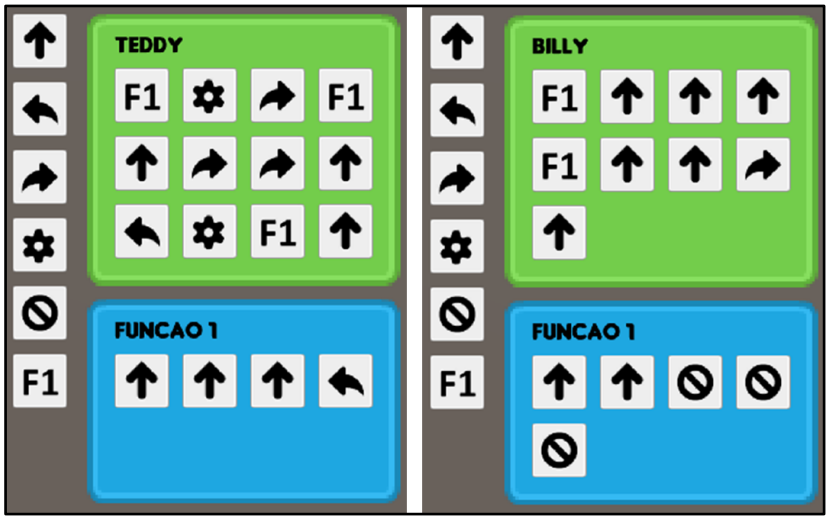
Esta seção está dividida em três subseções, sendo a 4.1 demonstra os testes de funcionalidades durante o desenvolvimento do projeto, a subseção 4.2 apresenta os resultados obtidos a partir de testes com usuários e por fim, na subseção 4.3 é apresentada uma comparação entre o jogo ProgramAR e os trabalhos correlatos.

4.1 TESTES DE FUNCIONALIDADES

Para realizar todos os testes da aplicação, visando seu funcionamento em ambos os sistemas operacionais, se utilizou os *smartphones* iPhone 13 Pro com o iOS 16.6 e Samsung A52s com Android 12, mas qualquer versão ou modelo de dispositivos móveis que sejam superior e contenham o ARKit ou ARCore, devem suportar a aplicação desenvolvida. Para realizar o desenvolvimento e os testes iniciais, utilizou-se o sistema operacional Windows 10 com uma configuração de 16 GB de RAM, com processador Intel Core i5 3.0 Ghz, com placa gráfica NVIDIA GeForce GTX 1660 e um SSD com velocidade de leitura de 545 MB/s.

Estes testes objetivaram encontrar o máximo de erros possível antes do jogo ser disponibilizado para outros usuários jogarem. Cada ação foi testada utilizando seu respectivo bloco com comando, na qual para cada astronauta foram inseridos diversos deles em sequência, podendo assim visualizar se todas as animações e interações com o cenário foram executadas corretamente. Um exemplo das sequências criadas para ambos os astronautas em testes, é o apresentado na Figura 13. Para este teste foi utilizado o nível 4 da página de funções, que para ser resolvido necessita de todos os blocos com comandos atualmente disponíveis e o painel de função.

Figura 13 - Exemplo de sequência correta de comandos utilizado em testes



Fonte: elaborado pelo autor (2023).

Outros testes também incluíram os *logs* de erro, na qual foram criadas lógicas incorretas intencionalmente, usando comandos de forma a gerar problemas de lógica. Como por exemplo, usar o comando de andar sendo que na frente do astronauta há um bloco bloqueando seu caminho, desta forma podendo testar se a tela de *log* de erros traria o erro corretamente. Quanto a rotina de validação da lógica criada pelo jogador, foram montadas diversas variações do algoritmo solução, todas percorrendo um caminho válido até o objetivo final, para garantir que todos os comandos estivessem sendo validados corretamente.

Com o intuito de encontrar possíveis problemas em aparelhos com configurações inferiores, foram realizados os mesmos testes com um MacBook mais antigo, neste momento sem o uso da RA, que executou o jogo com poucos quadros por segundo (*Frames Per Second* - FPS), capacidade que um dispositivo tem de renderizar e exibir imagens em um segundo. Isso ocasionou uma lentidão no jogo e portanto, um mal funcionamento na ação de andar dos astronautas, visto que a animação de andar foi implementada com o uso de *delays*, que utilizam os FPS's como base e que caso sejam muito baixos, fazem com que o astronauta não consiga realizar um ciclo completo (andar de um bloco até outro), ou seja, a partir de uma ação de andar o astronauta dificilmente conseguirá chegar no objetivo por não andar a distância correta e ainda gerar erros de colisão incorretamente. Junto a isso, devido ao uso da RA exigir um processamento maior do aparelho, isso indicou também que em dispositivos móveis com configurações baixas, o jogo poderá apresentar problemas na ancoragem dos cenários.

O principal problema encontrado, acabou por forçar uma mudança na forma em que a RA seria utilizada, na qual a ideia inicial era o jogador selecionar a posição para ancoragem do cenário apenas no menu inicial e o jogo manter guardado esta informação até que ele fosse fechado. Contudo, o `ARRaycastManager` não estava conseguindo manter esta informação entre as telas do jogo, perdendo a referência a cada troca e a partir disso, o jogador agora terá que ancorar os cenários sempre que trocar de nível ou reiniciá-lo.

4.2 TESTES COM USUÁRIOS

Com o objetivo de verificar a jogabilidade do jogo, foi elaborado e disponibilizado através do Google Forms um formulário a um grupo de 6 pessoas. O Quadro 6 apresenta as perguntas de perfil dos usuários, que possuem uma variação de idades entre 20 e 50 anos, na qual a grande maioria utiliza dispositivos móveis com frequência e já ouviram falar do conceito de RA.

Quadro 6 - Perfil do usuário

Ordem	Etapas
1	Idade
2	Você utiliza dispositivos móveis com qual frequência?
3	Você já ouviu falar em Realidade Aumentada?

Fonte: elaborado pelo autor (2023).

No Quadro 7 são apresentadas as perguntas relacionadas ao uso direto do jogo. Nelas nota-se que apenas 1 jogador teve dificuldades em fixar os cenários, todos leram os tutoriais e 4 acharam que são intuitivos e 2 não acharam, todos eles conseguiram compreender como utilizar os blocos com comandos e como funcionam as funções.

Quadro 7 - Perguntas sobre o uso do jogo

Ordem	Perguntas
1	Você conseguiu fixar os cenários com facilidade?
2	Você leu e achou os tutoriais intuitivos?
3	Você conseguiu entender como usar os blocos de ações?
4	Você conseguiu entender como funciona uma função?

Fonte: elaborado pelo autor (2023).

Para avaliar a construção do jogo como um todo, as perguntas apresentadas no Quadro 8 foram aplicadas, tendo pontuações que variam de 1 a 5, na qual representam uma escala de satisfação muito ruim a muito bom. No geral o jogo obteve uma avaliação boa, na qual a maioria das classificações ficaram entre 3 e 5. Para a usabilidade, 67% dos jogadores avaliaram como 4 e 33% avaliaram como 3, quanto as interfaces serem amigáveis, as avaliações ficaram entre 3 e 5 divididas com 33% dos jogadores cada. Sobre cumprir o objetivo de auxiliar no aprendizado da lógica, 84% concordaram que o jogo é uma opção para se trabalhar com este conceito. O uso da RA foi avaliada em 4 por 84% dos jogadores e em 3 por 16%, todos acharam interessante utilizar um jogo para abordagem da lógica de programação.

Quadro 8 - Perguntas de avaliação do jogo

Ordem	Perguntas
1	Como você classifica a usabilidade do ProgramAR em Geral?
2	Você acha que as interfaces do ProgramAR são amigáveis?
3	Você acha que o ProgramAR cumpriu seu objetivo de auxiliar no aprendizado da lógica?
4	Você achou intuitivo o uso da Realidade Aumentada no ProgramAR?
5	Você achou interessante o uso de um jogo para ensinar lógica?

Fonte: elaborado pelo autor (2023).

Todos os usuários respondentes já possuem algum conhecimento sobre lógica e algoritmos, pois trabalham com programação ou já tiveram contato em algum momento. Além das perguntas, também foi disponibilizado um espaço para comentários gerais, os quais foram utilizados para mapear possíveis melhorias e extensões para o jogo.

4.3 COMPARAÇÃO COM TRABALHOS CORRELATOS

O Quadro 9 apresenta uma comparação do jogo desenvolvido com os trabalhos correlatos. Conforme pode ser observado, todos os trabalhos abordam seus temas a partir do uso de jogos e possuem o ensino fundamental como público-alvo, com exceção do trabalho de Mello e Antoniazzi (2020) que focam também no ensino médio. Quanto a abordagem principal, os trabalhos de Saraiva (2022) e Kuntz (2020) trazem consigo o conceito de pensamento computacional que trabalha com diversos pilares que vão além do desenvolvimento da lógica, já Mello e Antoniazzi (2020) se aprofundaram no ensino da lógica e algoritmos, conceitos que estão fortemente relacionados.

O trabalho de Kuntz (2020) é o único que não trabalha com o uso da RA diretamente, tendo maior foco em trazer uma nova visão para o jogo Furbot desenvolvendo uma IUT. Os trabalhos de Saraiva (2022) e Mello e Antoniazzi (2020) utilizam marcadores para poderem renderizar os cenários em RA de seus jogos, sendo usada a plataforma Vuforia para realizar o pré-processamento das imagens utilizadas. Já o jogo ProgramAR não necessita de marcadores, utilizando o *framework* AR Foudation para permitir que os cenários em RA possam ser renderizados em qualquer superfície plana controlada.

Por fim, é possível ver que o jogo ProgramAR permite o acesso a uma gama maior de usuários, visto que foi testado e pode ser executado nos sistemas operacionais Android e iOS, os mais utilizados na atualidade, enquanto os trabalhos correlatos foram testados apenas na plataforma Android.

Quadro 9 - Comparativo com os trabalhos correlatos

	Saraiva (2022)	Mello e Antoniazzi (2020)	Kuntz (2020)	ProgramAR
Jogo educacional	Sim	Sim	Sim	Sim
Público-alvo	Ensino fundamental	Ensino fundamental e médio	Ensino fundamental	Ensino fundamental
Abordagem principal	Pensamento Computacional	Lógica e Algoritmos	Pensamento Computacional	Lógica
Método de desenvolvimento da lógica	Usando cartões com ações	Usando botões virtuais	Usando cartões com ações	Usando blocos com comandos
Utiliza Realidade Aumentada	Sim	Sim	Não	Sim
Dispensa o uso de marcadores	Não	Não	X	Sim
Ferramenta utilizada para trabalhar a RA	Vuforia	Vuforia	X	AR Foudation
Plataformas disponíveis	Android	Android	Android	Android e iOS

Fonte: elaborado pelo autor (2023).

Desta forma, o trabalho ProgramAR se mostra relevante como uma ferramenta de aprendizagem de lógica. O jogo busca ensinar utilizando uma maior interatividade com os cenários, dando aos jogadores uma maior liberdade ao permitir fixar o jogo em qualquer lugar e possibilitando andar ao seu redor para visualizar possíveis caminhos, armadilhas ou obstáculos, proporcionando assim uma maior imersão.

5 CONCLUSÕES

Com base nos testes realizados, o desenvolvimento do jogo ProgramAR proporcionou uma compreensão detalhada das complexidades e desafios inerentes à integração da Realidade Aumentada (RA) com a lógica de programação. Os testes extensivos com diferentes dispositivos revelaram questões cruciais de desempenho e usabilidade, destacando a importância da adaptação e otimização para uma variedade de configurações. Os desafios técnicos encontrados, como a adaptação da ancoragem dos cenários de RA, resultaram em mudanças significativas na estrutura do jogo, ilustrando a necessidade de flexibilidade e adaptação durante o processo de desenvolvimento.

Além disso, os testes com usuários proporcionaram uma visão valiosa sobre a receptividade e eficácia do jogo. A avaliação dos participantes revelou um entendimento positivo sobre a proposta de utilizar a RA como ferramenta para o aprendizado da lógica de programação. Ainda que tenham sido identificadas algumas dificuldades na conclusão de todos os níveis e na ancoragem dos cenários, a percepção geral foi favorável, destacando a eficácia do jogo como um recurso complementar para o ensino e aprendizado desses conceitos.

As análises detalhadas dos testes proporcionaram uma maior visão com relação a aprimoramentos futuros. Por mais que os cenários possam ser renderizados sem a necessidade de marcadores, ainda existem limitações quanto ao ambiente, como por exemplo locais com muito reflexo, terrenos irregulares ou variações de luminosidade. Também existe a limitação de hardware que pode causar mau funcionamento caso o aparelho utilizado possua uma configuração muito básica, na qual perderá facilmente o local em que o cenário foi ancorado e/ou executará o jogo com poucos FPS's, causando travamentos e incoerências na execução das ações programadas. Já com relação a possíveis extensões, durante os processos de desenvolvimento do jogo foram encontradas e mapeadas todas as descritas abaixo:

a) criar mais blocos com comandos, permitindo ensinar outros conceitos de lógica de programação como por exemplo laços de repetição ou estruturas de seleção;
b) implementar a funcionalidade multijogador, na qual os jogadores teriam acesso ao mesmo cenário, porém, cada um com seu dispositivo e programando seu próprio astronauta, a fim de chegarem juntos no destino;
c) criar um sistema de pontuação baseado na quantidade de blocos utilizados, a fim de incentivar os jogadores a criarem lógicas com menor grau de complexidade e de fácil entendimento;
d) melhorar o sistema de ancoragem, visando evitar que o jogador tenha que ancorar o cenário cada vez que passe de nível ou precise reiniciá-lo;
e) deixar os tutoriais mais intuitivos, substituindo os textos por vídeos curtos.

REFERÊNCIAS

BATISTA, Riann Martinelli; FONSECA, Alexandre Ramos; HORTA, Euler Guimarães. Programação em Blocos: impacto de um projeto de extensão executado em Escolas públicas de Diamantina/MG. **Informática na educação**: teoria & prática, Porto Alegre, v.23, n.2, maio/ago. 2020. Disponível em: https://seer.ufrgs.br//InfEducTeoriaPratica/article/view/96741. Acesso em: 18 jun. 2023.

ELVAS, Filipe Barbosa da Cunha Mendes. **Realidade Aumentada Aplicada a Panoramas Táticos**. 2018. Dissertação (Mestrado em Ciências Militares Navais) – Escola Naval, Alfeite, 2018. Disponível em: https://comum.rcaap.pt/handle/10400.26/25097. Acesso em: 23 mar. 2023.

FURB. **FURBOT**. 2022. Disponível em: https://furbotldtt.wixsite.com/my-site-1. Acesso em: 19 dez. 2023.

KIRNER, Claudio; SISCOUTTO, Robson. Realidade Virtual e Aumentada: Conceitos, Projeto e Aplicações. *In*: **Symposium on Virtual and Augmented Reality**, 2007, Petrópolis. Porto Alegre: SBC, 2007. p. 2-292. Disponível em: http://de.ufpb.br/~labteve/publi/2007_svrps.pdf. Acesso em: 7 abr. 2023.

KUNTZ, Jonathan Michels. **Interface de usuário tangível para trabalhar com o pensamento computacional no FURBOT**. Trabalho de Conclusão de Curso (Bacharel em Ciência da Computação) - Universidade Regional de Blumenau, Blumenau, 2020. Disponível em: https://www.furb.br/dsc/arquivos/tccs/monografias/2020_1_jonathan-michels-kuntz_monografia.pdf. Acesso em: 16 jun. 2023.

MELLO, Luiz Filipe Durgion de; ANTONIAZZI, Rodrigo Luiz. Jogo com utilização de realidade aumentada voltado para o desenvolvimento lógico aplicado ao ensino fundamental e médio. **RevInt**: Revista Interdisciplinar de Ensino, Pesquisa e Extensão, Cruz Alta, v.8. 2020. Disponível em: https://revistaeletronica.unicruz.edu.br/index.php/revint/article/view/386. Acesso em: 13 mar. 2023.

MORAIS, Ceres Germanna Braga; MENDES NETO, Francisco Milton; OSÓRIO, António José. Dificuldades e desafios do processo de aprendizagem de algoritmos e programação no ensino superior**:** uma revisão sistemática de literatura. **Research, Society and Development**, [*s. l.*], v.9, n.10, 2020. Disponível em: https://rsdjournal.org/index.php/rsd/article/view/9287. Acesso em: 23 mar. 2023.

SANTOS, Fabiano. **Lógica de Programação**. 1. Ed. Rio de Janeiro: Editora SESES, 2015. Disponível em: http://hood.com.br/new/filegator/repository/Introdu%C3%A7%C3%A3o%20%C3%A0%20Programa%C3%A7%C3%A3o/10%20Livro%202.pdf. Acesso em: 20 jun. 2023.

SARAIVA, Francisco Manuel Vital. 2022. Dissertação (Mestrado em Engenharia de Informática) – Escola de Engenharia, Universidade do Minho, Minho, 2022. **Building a game with Augmented Reality:** for training computational thinking. Disponível em: https://repositorium.sdum.uminho.pt/handle/1822/83352. Acesso em: 20 de mar. 2023.

SOUZA, Draylson Micael; BATISTA, Marisa Helena da Silva; BARBOSA, Ellen Francine. Problemas e Dificuldades no Ensino e na Aprendizagem de Programação: Um Mapeamento Sistemático. **Revista Brasileira de Informática na Educação**, São Paulo, v.24, n.1, 2016. Disponível em: http://ojs.sector3.com.br/index.php/rbie/article/view/3317. Acesso em: 23 mar. 2023.

SOUZA, Franciely Alves de; FALCÃO, Taciana Pontual; MELLO, Rafael Ferreira. O Ensino de Programação na Educação Básica**:** Uma Revisão da Literatura. *In*: SIMPÓSIO BRASILEIRO DE INFORMÁTICA NA EDUCAÇÃO, 32., 2021, Recife. **Anais** [...]. Recife: UFRPE, 2021. p. 1265-1275. Disponível em: https://sol.sbc.org.br/index.php/sbie/article/view/18148. Acesso em: 23 mar. 2023.

TORI, Romero; HOUNSELL, Marcelo da Silva; KIRNER, Claudio. Realidade Aumentada. *In*: **Introdução a Realidade Virtual e Aumentada**. 3. Ed. Porto Alegre: Editora SBC, 2020. Disponível em: https://sol.sbc.org.br/livros/index.php/sbc/catalog/book/66. Acesso em: 20 jun. 2023.

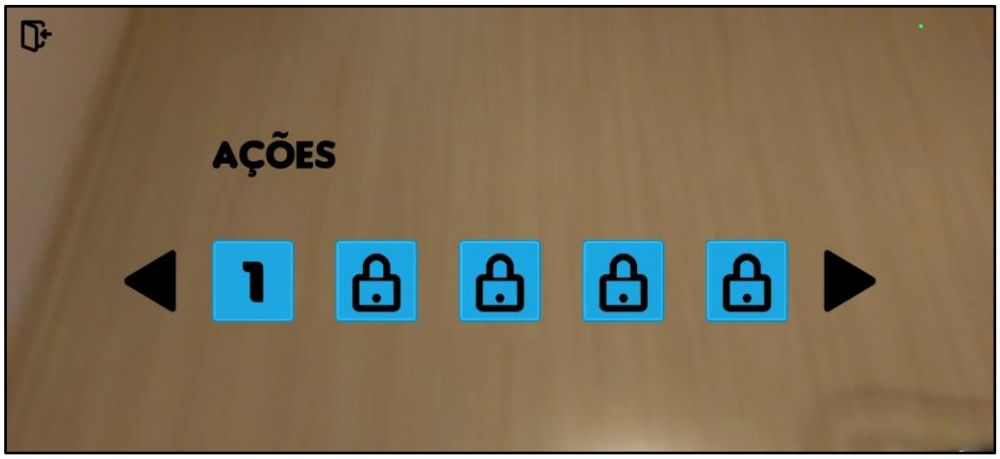
UNITY. **AR Foundation**. 2023. Disponível em: https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@5.0/manual/index.html. Acesso em: 17 abr. 2023.

XAVIER, Gley Fabiano Cardoso. **Lógica de Programação**. 13. Ed. São Paulo: Editora Senac, 2014. Disponível em: https://books.google.com.br/books?hl=pt-BR&lr=&id=lxW8EAAAQBAJ&oi=fnd&pg=PP5&dq=l%C3%B3gica+de+programa%C3%A7%C3%A3o&ots=47IqhldzV9&sig=4Kf4Fk5Wq3_eqdBDKlvUpNsvD0A#v=onepage&q&f=false. Acesso em: 20 jun. 2023.

APÊNDICE A – VISÃO GERAL DAS TELAS DO JOGO

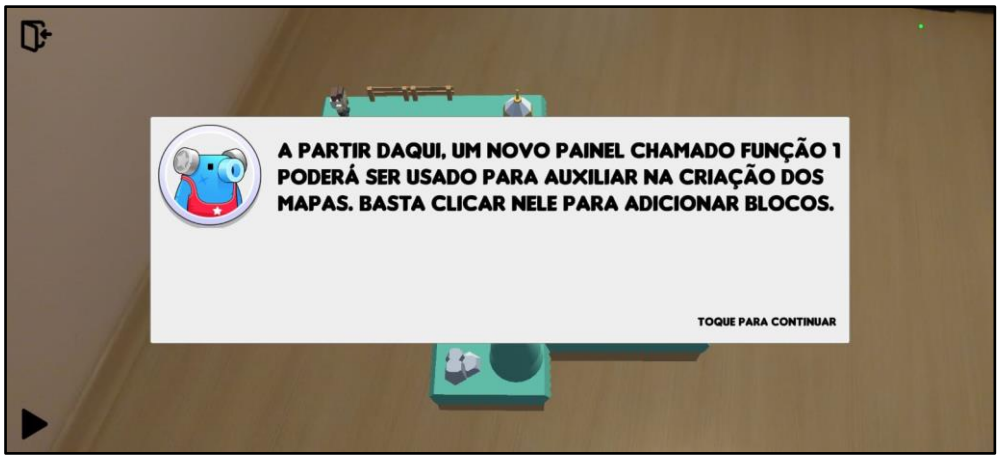
Este apêndice traz uma visão geral das telas do jogo, juntamente a utilização da visualização em Realidade Aumentada. Na Figura 14, é apresentada a tela de seleção de níveis, na qual o jogador possui as páginas de Ações e Funções para selecionar um cenário. Com a Figura 15 se tem um exemplo de tela de tutorial que se abre ao posicionar um cenário, trazendo consigo uma breve explicação do que será proposto no nível selecionado. Por fim, na Figura 16 é apresentada a visão dos cenários do jogo, tendo o astronauta selecionado destacado por uma borda vermelha, juntamente ao seu correspondente painel de programação e função, sua tela de *log* de erros e os blocos disponíveis para o nível.

Figura 14 - Tela de seleção de níveis



Fonte: elaborado pelo autor (2023).

Figura 15 - Tela de tutorial



Fonte: elaborado pelo autor (2023).

Figura 16 – Visão dos cenários, tela de programação e tela de logs

Fonte: elaborado pelo autor (2023).